

Github Copilot – Python

Website Development With Python and Copilot

Environment:

The bootcamp is based on a Linux environment and VS Code. The instructor is using Ubuntu 22 with Python 3 (3.10.12). You will also need a browser and Postman.

Before proceeding open a terminal and execute this command: **python3 --version**

If you see something like **3.10.xx** then you can proceed. If not, try updating your OS and install Python for your system.

Optional

Type the command **python3** and hit the enter key. If you see something like:

```
axle@pc0482:~$ python3
Python 3.10.12 (main, Feb  4 2025, 14:57:36) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

Then you are in the right environment.

To exit the interpreter, type **exit()** or press **Ctrl+D**.

Part 01: Getting Started with Python

Open VS Code and then open the Copilot Chat window. If you just want to learn the basics you can use prompts like the following:

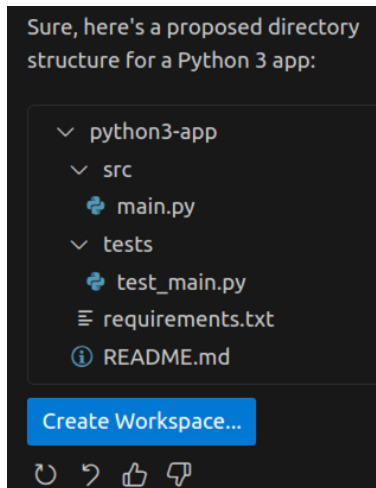
**I want to learn Python, help me get started, or,
Give me a simple exercise to learn Python.**

We will begin writing Python programs using a boilerplate application written for us by Copilot, here is my prompt:

I want to start developing an app in Python3, help me generate and build a boilerplate application

Note, there is a high probability that you will NOT get the same results as I got in this document, that is how the tool works. This is expected behavior.

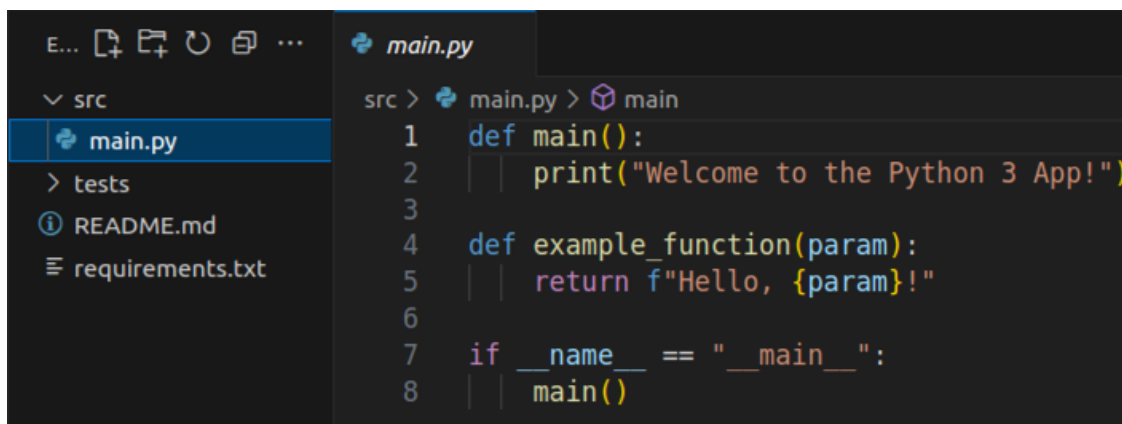
This is how Copilot responded:



At this point, all I have to do is click that Create Workspace button directly in the Chat window and the project folder with a name that Copilot chose will be created for me. You will have to choose a parent folder from the file system. Once you do that you will be asked to open the current app or open a new VS Code window.

Note, VS Code will invite you to install the Python extension if you don't already have it.

Here I am showing you the contents of the **main.py** file as well as the environment that Copilot built for me:



So, now if you want, you can prompt about this file, how to execute it: **how can I execute the main.py file**

It should respond by telling you to open a terminal window and execute the file using a command similar to this:

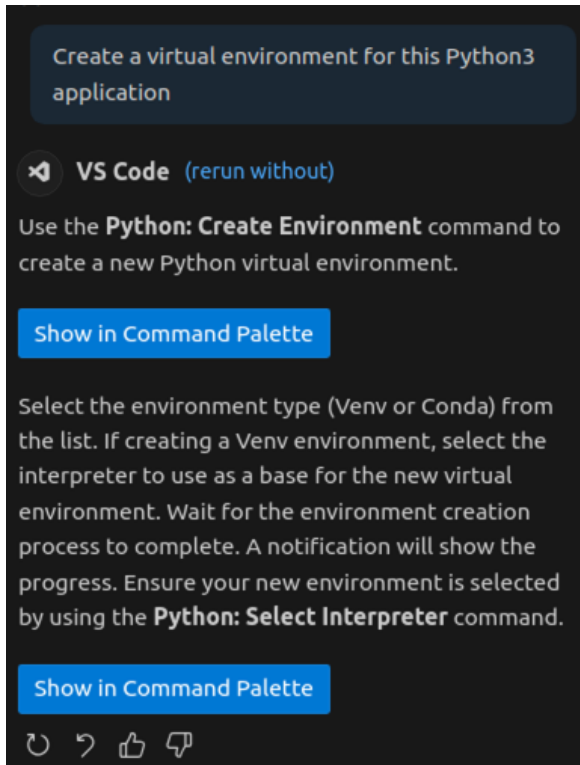
python3 src/main.py

Notice the path

Finally for this section, we should create a virtual environment for this application. Here is the prompt:

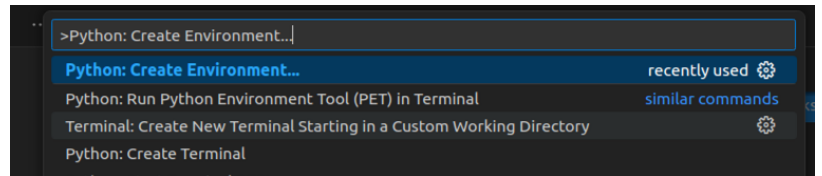
Create a virtual environment for this Python3 application

This is how my prompt responded:

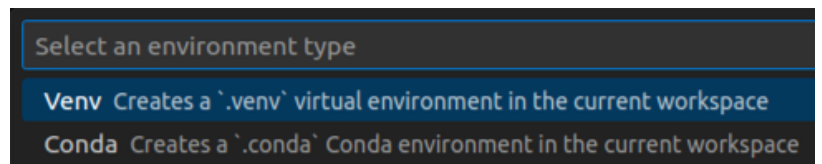


This part is heavily dependent on VS Code since we are in that environment.

This is what I saw after clicking on the Show in Command Palette blue button:

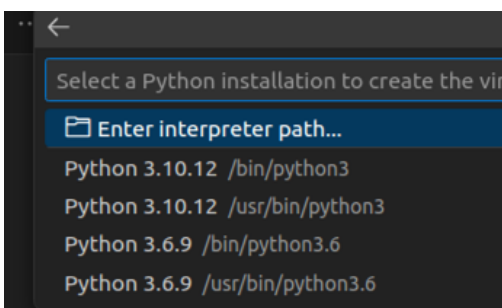


Click the Python: Create Environment option, then:

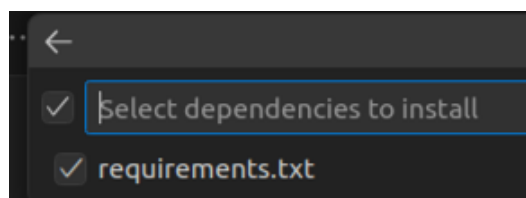


Choose the **venv** option which is Python's built-in module for creating **isolated Python environments**. It avoids conflicts with other Python folders and the system's global Python installation.

You will then be presented with a few different interpreters, choose the one corresponding to your installed Python version, in my case it is 3.10.12



The final question has to do with dependencies, in my case I have the option of the requirements.txt file. If you do not see anything here, just click the blue OK button



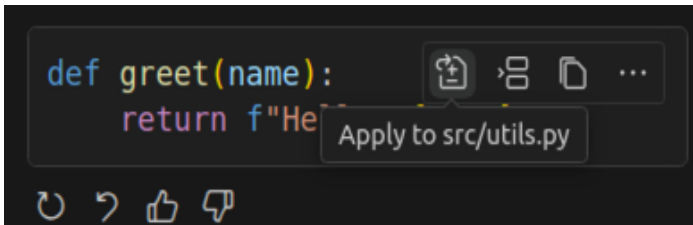
After all these options, you should have a .venv file in your project folder.

Part 2

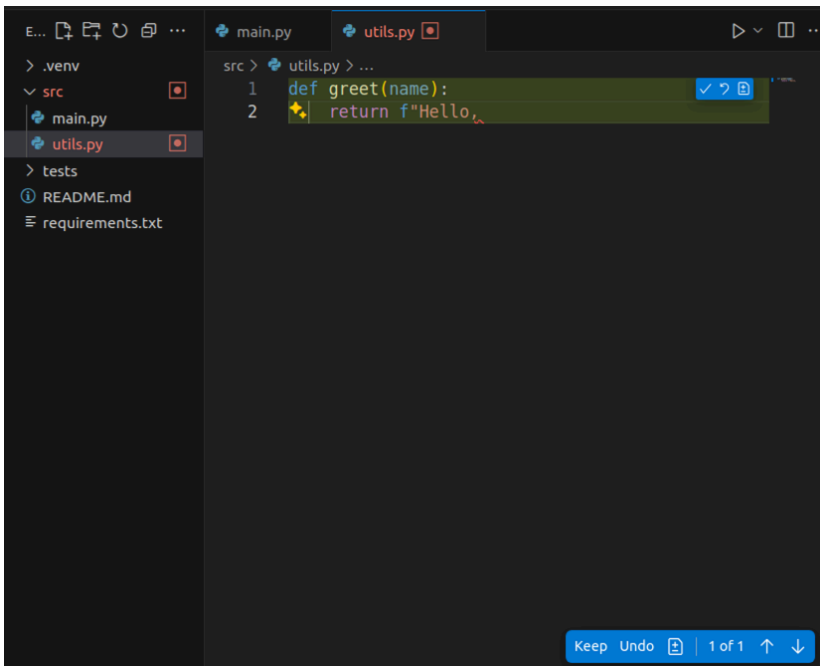
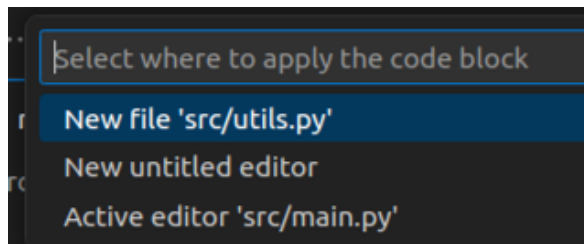
The next logical step is to create modules and then import them into the `main.py` file. This is a fundamental concept in software development, promoting code reusability and maintainability.

I want to start using modules, create a new file called `utils.py` which I will then import into the `main.py` file

In my case, Copilot responded with this code. Notice the three icons at the top of the response window. The left icon is the most used. As I hover over it, the pop-up reads, **Apply to `src/utils.py`**. This means that it wants to create a new file for me, so just click it and it will take you to the top of VS Code where you can proceed to create this new file.

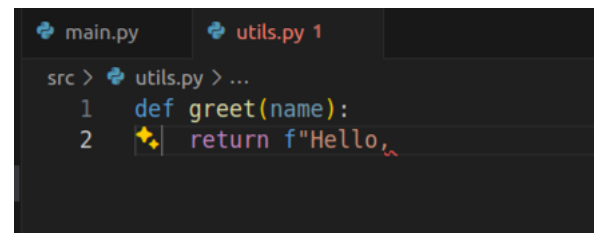


In our case we will choose the first option, which will create a new `.py` file for us in the `/src` folder:



You will then have to confirm that this is what you want by clicking the blue **Keep** button at the bottom right of the editor window.

Undo, will of course not save the `utils.py` file.



Notice the double starry looking icon to the left of line 2. This is called the **sparkle** icon

and when it appears, usually it will help you write the next line of code. In my case here, it seems that part of the original response is missing. Either use the sparkle icon to help replace the missing part or use one of the three icons mentioned earlier to copy the code and replace the current block of code with the entire correct block. The third icon to the right of the three icons that appear will be the copy icon. These are the icons that appear above the response in Copilot Chat window.

Sometimes when Copilot detects obvious errors it will try to help you immediately:

```
main.py  utils.py 1
src > utils.py > ...
1 def greet(name):
2  ✨ return f"Hello, {name}!"
```

In this case here, Copilot is suggestion that I enter the code in italics. This is called **ghost text**. To accept, just hit the tab key on the keyboard.

The red squiggly line is from VS Code, so nothing to do with GitHub Copilot.

At this point, we now have two files main and utils. Ask Copilot how you can consume the utils file from main:
How can I call this utils file from the main file

This is how Copilot responded in my case:

```
To use functions from utils.py in your main.py ,
import the module at the top of your main.py
file. For example:

from utils import greet

def main():
    print("Welcome to the Python 3 App")
    print(greet("User"))

if __name__ == "__main__":
    main()

Make sure both main.py and utils.py are in
the same directory ( src ). This allows you to
import greet directly from utils.
```

So now we know that the import keyword exists in Python just like in many other languages.

Once we import the utils module, we simply use the methods, in this case the method name is `greet()`.

Here is the code in the `main.py` file so far. I added a comment to the original line 4:

```
from utils import greet

def main():
    # print("Welcome to the Python 3 App!")
    print(greet("User"))
```

If you run this program now, it will print “Hello, User!” in the console window.

Lets now experiment with modules that ship with Python3. We will use the `utils.py` file to export a random function. Remove all the code from the `utils.py` file and prompt Copilot. In my case `utils.py` is the file in context:

Remove all the code from this file and create a function to return a random number

This is the response:

```
import random

def get_random_number():
    return random.randint(1, 100)
```

Notice that it imported `random`, which is a Python module. Also the random number will be between 1 and 100

Now back on the `main.py` file, prompt to access the modified `utils.py` file:

How can I use or consume the `utils.py` file and new function from `main.py`

```
from utils import get_random_number

def main():
    print(f"Random number: {get_random_number()}")
```

Notice that in this case, the import statement is slightly different.

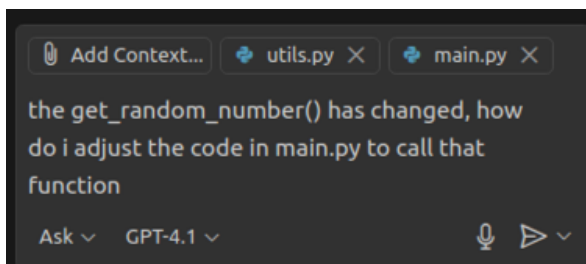
Lets change the `utils` to accept a parameter and this numeric value will be the upper limit of the random number:

I want to change this `get_random_number` function to accept a parameter, a number. Use that number to return a random number greater than zero but less than the number passed in as a parameter

```
import random

def get_random_number(max_number):
    return random.randint(1, max_number)
```

Now, I want to add both files to the context of my next prompt. Choose a file and that file becomes automatically goes into context. Then, just click and drag one or more files that you want the next prompt to consider in it's response. Here I have both the `utils.py` and the `main.py` files in context:



Notice that my next prompt is in the image.

This is the suggested code for the function call:

```
from utils import get_random_number

def main():
    max_value = 10 # set this to any positive integer > 1
    print(f"Random number: {get_random_number(max_value)}")
```

Part 3

In this part we will attempt to learn specific concepts in Python like classes. Create a new file called `learn.py` using VS Code. No need for a prompt, just create the file the old fashioned way.

Here is my first prompt about classes in Python3:

I want to learn about classes and consuming them, can you generate an example of a class with methods to add, subtract, multiply and divide numbers.

This is the suggested code for the class called Calculator:

```
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b
...
if __name__ == "__main__":
    calc = Calculator()
    print("Add: 2 + 3 =", calc.add(2, 3))
    print("Subtract: 5 - 2 =", calc.subtract(5, 2))
...
```

Note, I did not enter the entire file here, full file in download folder.

This is my next prompt:

How can I interact with this Calculator class from main.py

This is the response:

```
from learn import Calculator

def main():
    calc = Calculator()
    print("Add: 2 + 3 =", calc.add(2, 3))
    print("Subtract: 5 - 2 =", calc.subtract(5, 2))
    print("Multiply: 4 * 3 =", calc.multiply(4, 3))
    print("Divide: 10 / 2 =", calc.divide(10, 2))
```

In the `learn.py` file there is the keyword “self” in almost all functions. Lets ask Copilot about this:

In this code I see the word "self" as a parameter, what is the purpose of this code and is it mandatory

Copilot should respond with an explanation. Think of this “self” keyword like the “this” keyword in C-style languages like JavaScript.

Part 4

In this part we will attempt to work with files in the file system.

In the /src folder, create a new folder called data and insert the db.json file provided.

In the utils.py file, add code that would read the db.json file located in a folder called data which is directly below the src folder

This is the suggested code for the class called Calculator:

```
import random
import json
import os

...

def read_db_json():
    data_path = os.path.join(os.path.dirname(__file__), 'data',
'db.json')
    with open(data_path, 'r') as json_file:
        data = json.load(json_file)
    return data
```

Note, I did not enter the entire file here, this is just part of the entire file.

Your next prompt may be:

How can I test the read_db_json function from main.py

This is the suggested code:

```
# Test read_db_json
try:
    db_data = read_db_json()
    print("db.json contents:", db_data)
except FileNotFoundError:
    print("db.json file not found.")
except Exception as e:
    print("Error reading db.json:", e)
```


The next prompt I am sending is to add a function that will add a new employee document (record). Without refining the prompt too much this is what I am asking:

In the utils.py file, add a new function that will add a new employee object to the db.json file

In my case, it added an add_employee() function at the bottom of the utils.py file and it seems to be using the append() function to add a new_employee. The problem is that it never defined the new_employee!

```
def add_employee_to_db(employee):  
    """  
    Adds a new employee object to the db.json file.  
    The db.json file should contain a list of employee objects.  
    """  
    data_path = os.path.join(os.path.dirname(__file__), 'data',  
'db.json')  
    # Read existing data  
    try:  
        with open(data_path, 'r') as file:  
            data = json.load(file)  
    except (FileNotFoundError, json.JSONDecodeError):  
        data = []  
  
    # Add the new employee  
    data.append(employee)  
  
    # Write back to the file  
    with open(data_path, 'w') as file:  
        json.dump(data, file, indent=4)
```

Here is my next prompt:

On line 27 we add a new employee, can you define a new employee in this file to be added based on the data in the db.json file

This is how it responded:

```
        data = json.load(json_file)  
        return data  
  
# Define a new employee based on the structure in db.json  
new_employee = {  
    "id": "4",  
    "username": "John",  
    "password": "1234"  
}  
  
def add_employee(new_employee):
```

You can prompt the tool to call this new function from main:

Call this new function `add_employee` from main

This is how it responded:

```
from utils import get_random_number, read_db_json, add_employee_to_db
...

# Add a new employee
new_employee = {
    "id": get_random_number(1000),
    "username": "Alice",
    "password": "securepass"
}
add_employee_to_db(new_employee)
print("Added new employee:", new_employee)

... rest of file
```

Amazing! I was worried that the new employee would just be added to the end of the `db.json` file (after the last curly brace), but instead, the code added the new employee exactly where it needed to be:

```
    "password": "1234"
},
{
    "id": "4",
    "username": "John",
    "password": "1234"
}
```

Note, it did add the **dict** check for the type of file I am using:

```
if isinstance(data, dict):
    # If your db.json is a dict with a key like "employees"
    data = data.get("employees", [])
```

This was added to the `read_db_json()` function in `utils.py`

`dict` is short for dictionary

Part 5

In this part we will attempt to build a working API using Python. I will switch to **Agent** mode for this part.

I want to use this project to start learning about building APIs where do you suggest I start. Should I create a new file? I want to serve the data that I already have in the db.json file which is in the data folder which itself is in the root of this project.

This is the response:

A great way to start learning about building APIs in Python is to use a lightweight web framework like FastAPI or Flask. For your use case—serving the data from your existing db.json file—FastAPI is a modern, beginner-friendly choice that provides automatic documentation and is easy to set up.

Here's what I recommend:

Install FastAPI and an ASGI server (like uvicorn).
Create a new file (e.g., api.py) in your src/ directory.
In this file, set up a simple FastAPI app that reads from db.json and serves the data at an endpoint (e.g., /data).

Run the API and test it in your browser or with curl.

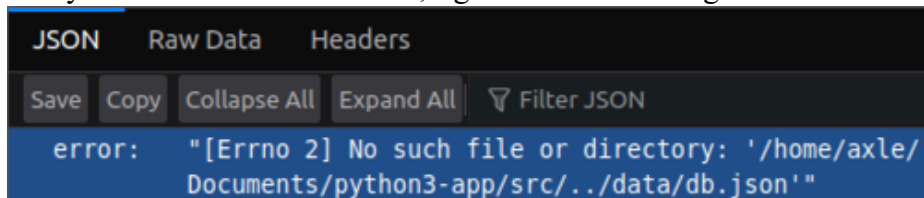
Would you like me to set up the initial FastAPI API file and install the required dependencies for you?

Notice that it talked about the main server software but it also added that dependencies to those software may be necessary. Lets proceed with Flask, since that seems to be the more popular web framework for Python.

I prefer Flask, proceed with Flask instead. Do a step by step installation so that I can track what is being installed.

Since I asked for a step by step installation, the process will ask several times for your permission to proceed. Once the installation is complete, it will ask to run the server in the background. It will give you a port number from which you can see the data in the db.json file.

In my case I did not see the data, I get this error message in the browser:



After some back and forth, Copilot finally got the paths right and it updated the path in the api.py file and I was able to see the data

Here is the entire api.py file that Copilot added to my src directory:

```
from flask import Flask, jsonify
import json
import os

app = Flask(__name__)

def get_db_path():
    # Adjust path to point to src/data/db.json
    return os.path.join(os.path.dirname(__file__), 'data', 'db.json')

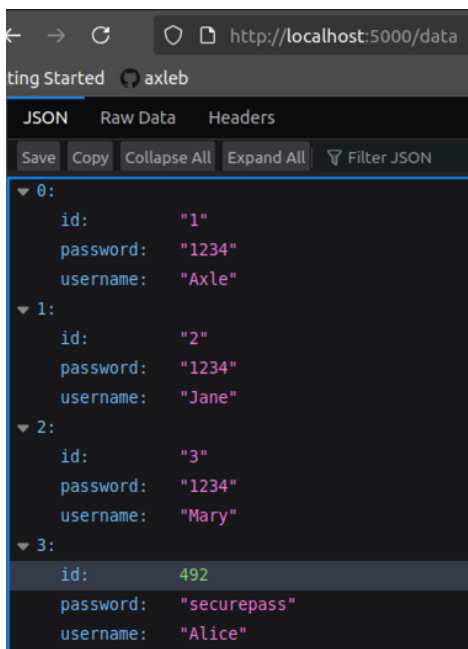
@app.route('/data', methods=['GET'])
def serve_data():
    db_path = get_db_path()
    try:
        with open(db_path, 'r') as f:
            data = json.load(f)
            return jsonify(data)
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)
```

To execute this file and see the data in the browser, it suggested this command:

```
/home/axle/Documents/python3-app/.venv/bin/python src/api.py
```

Once you start the server, it will respond with a URL that you can visit:



I then asked Copilot about POSTing data to the json file. This is how it responded:

A POST endpoint has been added to your API. You can now send a JSON payload to /data to add a new document to db.json.

Example usage:

Send a POST request with JSON data to <http://localhost:5000/data>.

If you need an example of how to test this with curl or Python, let me know!

The "Import 'flask' could not be resolved" warning is likely due to your editor not detecting the virtual environment—your code will still run if Flask is installed.

This is the code that Copilot added to the api.py file:

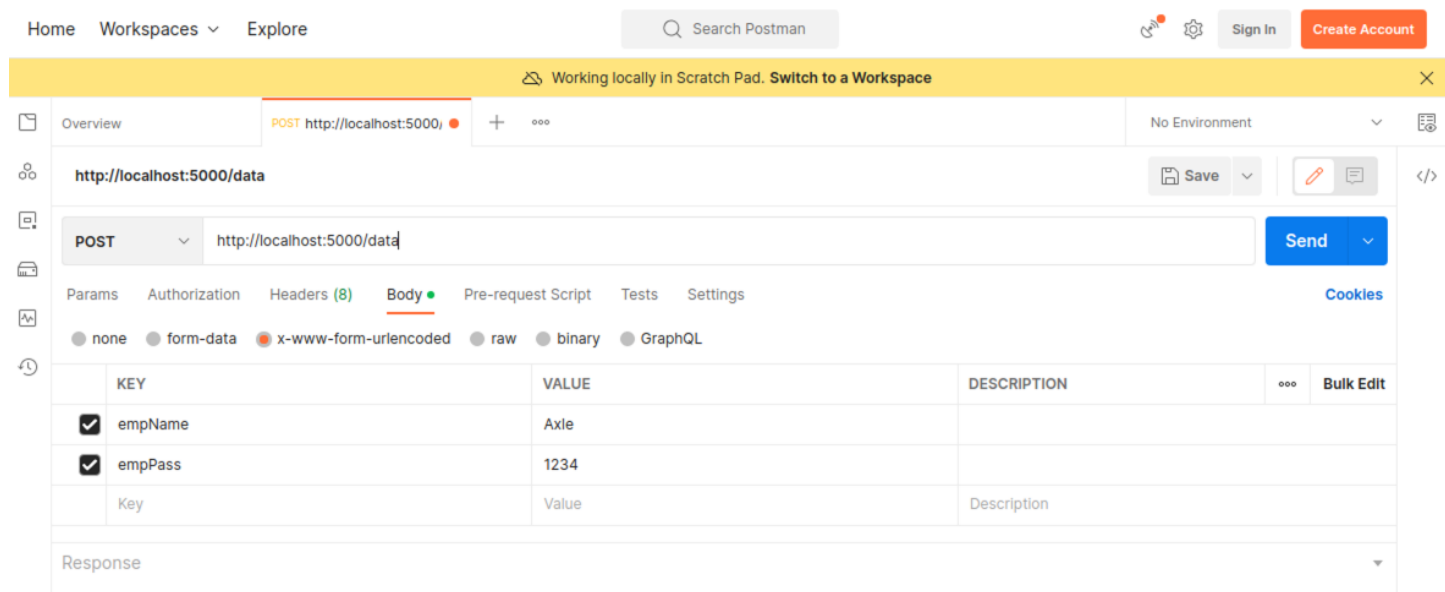
```
# POST endpoint to add a new document
from flask import request
@app.route('/data', methods=['POST'])
def add_document():
    db_path = get_db_path()
    try:
        new_doc = request.get_json()
        if not new_doc:
            return jsonify({'error': 'No JSON payload provided'}), 400
        # Read existing data
        with open(db_path, 'r') as f:
            data = json.load(f)
        # Append new document
        if isinstance(data, list):
            data.append(new_doc)
        else:
            return jsonify({'error': 'Database is not a list'}), 500
        # Write back to file
        with open(db_path, 'w') as f:
            json.dump(data, f, indent=2)
        return jsonify({'message': 'Document added', 'document':
new_doc}), 201
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

Note, to start up the server, you must do so from the .venv environment. For example, in my case it is:
`/home/axle/Documents/python3-app/.venv/bin/python src/api.py`

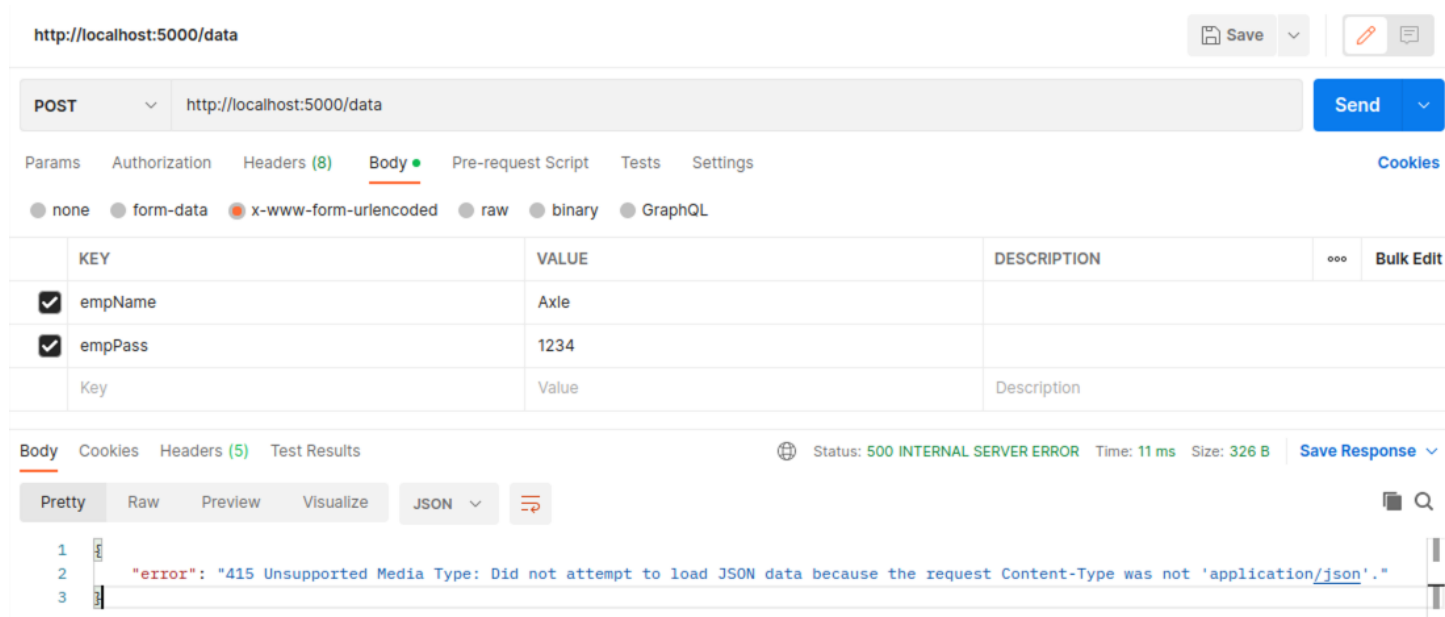
To access the data endpoint: `http://localhost:5000/data`

To add data to the .json file you must make a POST request, so you cannot use a browser. Notice however, Copilot did suggest that Python can be used to test this endpoint. Lets try Postman first.

If you have access to Postman, this is how you can post your new document (record):



I did get an error as shown below:



I then copied the entire error line and used it in my prompt to Copilot:

I used Postman to test the endpoint and i got an error: "error": "415 Unsupported Media Type: Did not attempt to load JSON data because the request Content-Type was not 'application/json'."

This is the response from Copilot:

This error means the request did not have the correct Content-Type header. In Postman, make sure to:

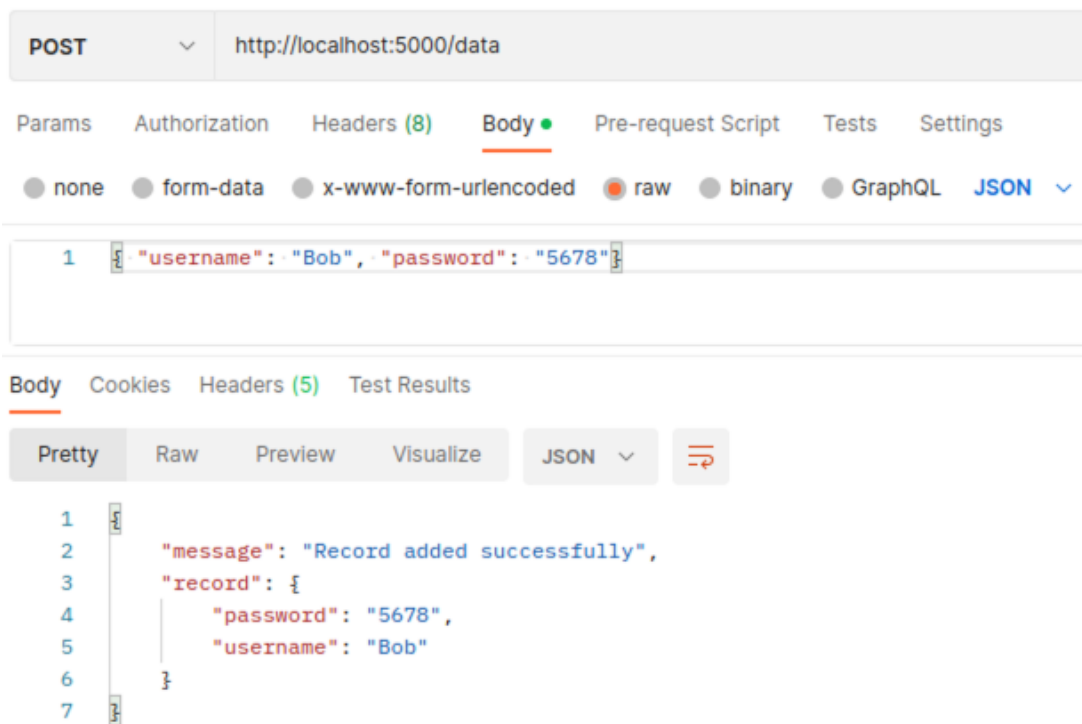
Set the request type to POST.

In the "Body" tab, select "raw" and choose "JSON" from the dropdown.

Ensure the header Content-Type: application/json is present (Postman usually adds this automatically when you select JSON).

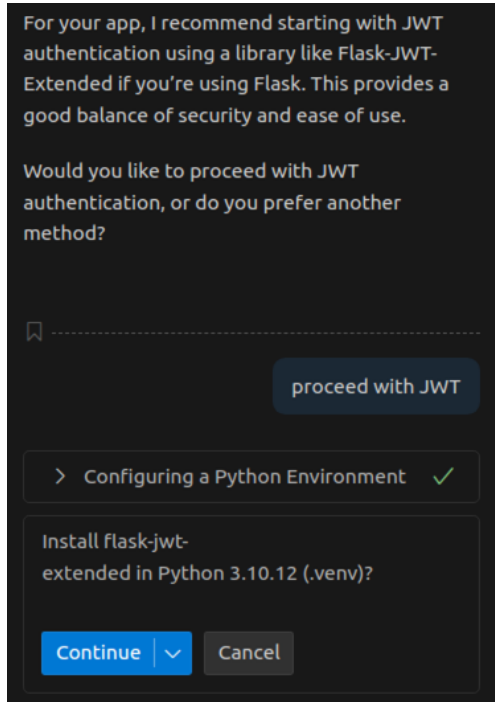
If you do this, the endpoint will accept your request. Let me know if you need a screenshot or more help with Postman!

I was not able to use the built-in form in Postman, but this worked:



Notice that I did not include an **id** as I thought the Flask software would add one for me, but it did not.

Part 6



In this part let's add security to our endpoints. To save time, Copilot did suggest about four options but adding tokens with a package like JsonWebToken was the suggested option. I chose that option and asked Copilot to add this feature:

As you can see from the image to the left, Copilot is asking me to add the package in the environment I created at the start of this bootcamp.

Below you will see the code that Copilot added in the api.py file.

```
# Login endpoint to issue JWT tokens
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    # Simple authentication: check against db.json
    db_path = get_db_path()
    try:
        with open(db_path, 'r') as f:
            users = json.load(f)
            user = next((u for u in users if u['username'] == username and
u['password'] == password), None)
            if not user:
                return jsonify({'error': 'Invalid credentials'}), 401
            access_token = create_access_token(identity=username)
            return jsonify(access_token=access_token)
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```


Copilot also suggested we test our new endpoint, this is how it responded:

You can now test your API by:

Sending a POST request to /login to get a token.
Including the token in the Authorization header (Bearer <token>) when accessing /data endpoints.
Notice that it talked about the main server software but it also added that dependencies to those software may be necessary. Lets proceed with Flask, since that seems to be the more popular web framework for Python.

My next prompt is to understand exactly how to test this endpoint, so I asked how to configure Postman to make this call:

How can I use Postman to test this endpoint

This is how it responded:

To test your JWT-protected endpoints with Postman, follow these steps:

1. Login to get a token:

Set method to POST and URL to <http://localhost:5000/login>

In the Body tab, select "raw" and "JSON", then enter:

```
{ "username": "Alice", "password": "securepass" }
```

Click "Send". The response will include an access_token.

2. Use the token to access protected endpoints:

Copy the access_token from the login response.

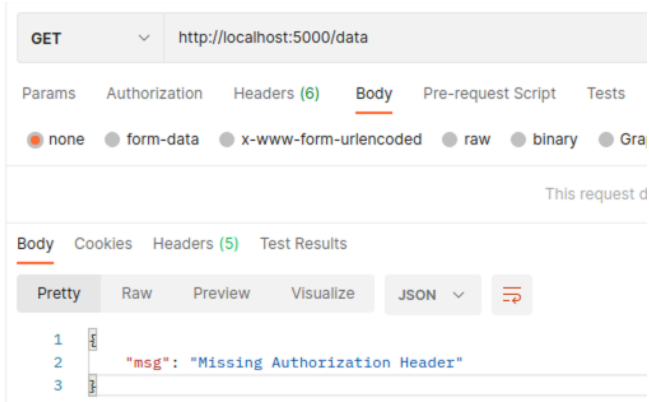
For GET or POST requests to http://localhost:5000/data, go to the "Authorization" tab.

Select "Bearer Token" and paste the token.

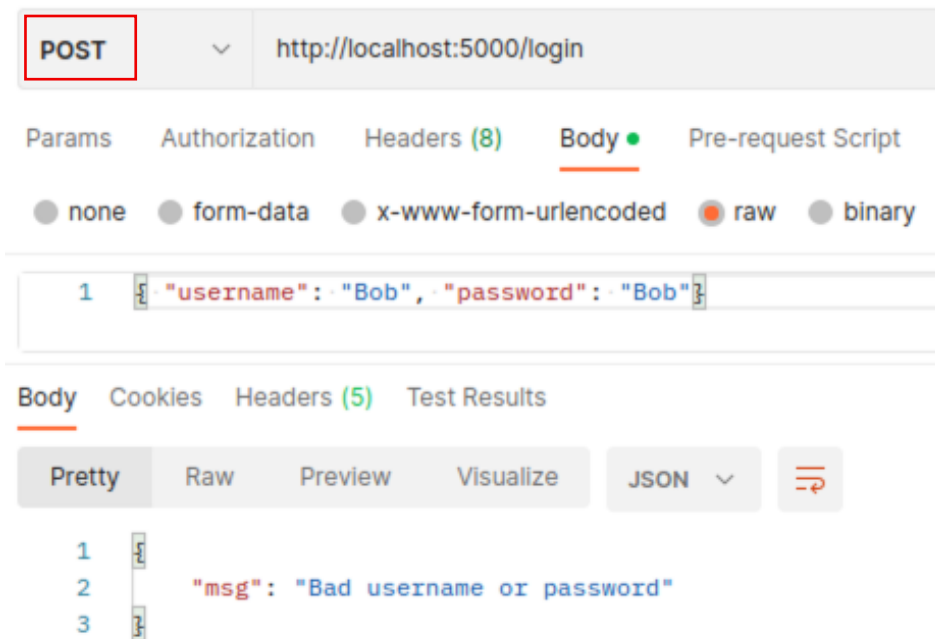
For POST /data, set the body as raw JSON for your new document.

This will authenticate your requests using JWT in Postman.

If we try a simple GET request in Postman, so no credentials, this is how it responds now:

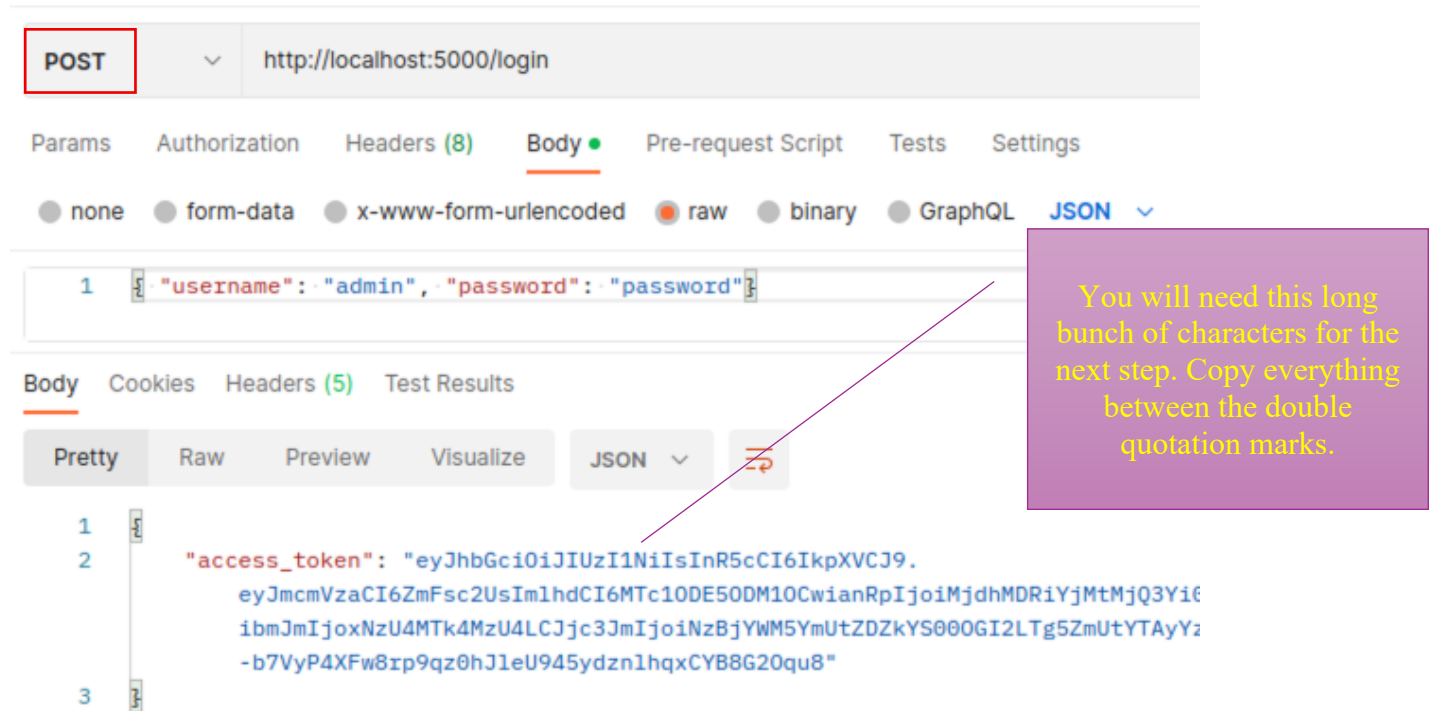


Let's try to login with incorrect credentials:



Now with the proper credentials:

`http://localhost:5000/login`



The screenshot shows a REST client interface. The top bar indicates a **POST** request to `http://localhost:5000/login`. Below the bar, the **Body** tab is selected, showing a JSON payload: `{ "username": "admin", "password": "password" }`. The response body is also shown in the **Body** tab, displaying a long `access_token` string. A callout box with a purple background and yellow text points to the token, stating: "You will need this long bunch of characters for the next step. Copy everything between the double quotation marks."

Note, in this image part of the token is clipped out, the token will change each time you login, so it is only temporary.

Anyway, with the token you can now try to access the `/data` endpoint:

The screenshot shows the Postman interface for a GET request to `http://localhost:5000/data`. The **Authorization** tab is selected, showing the **Bearer Token** type. A warning message states: "Heads up! These parameters hold sensitive data. To keep this data secure while working in a recommend using variables. [variables](#)". A purple box with the text "Paste token in this box" points to the **Token** input field, which contains the token `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...`. The **Body** tab is also selected, showing a JSON response in the **Pretty** view. The response is a list of three employees:

```
1 {
2   "employees": [
3     {
4       "id": "1",
5       "password": "1234",
6       "username": "Axle"
7     },
8     {
9       "id": "2",
10      "password": "1234",
11      "username": "Jane"
12    },
13    {
14      "id": "3",
15      "password": "1234",
16      "username": "Mary"
17    }
18  ]
19 }
```

At the top right, the status is **200 OK** and the time taken is **9 m**.

Things to note here.

1. Once you go to `/login` and got the token (the really long weird looking string) you must now change the Postman request a little.
2. In Postman, change the request to a GET and the URL to `/data`.
3. In the Authorization tab, choose the type as **Bearer Token**.
4. Paste the token in the space provided for tokens
5. Click the blue Send button at the top right corne of Postman UI.

Part 7

So far we have been using Postman to test our endpoints. I wanted to see if I can test these endpoints without going to an external technology, so I asked Copilot:

Can I test these endpoints using just the Python stack and not having to go to Postman

Of course it responded in the affirmative and it wanted to know if I wanted scripts generated for this purpose. Copilot responded by adding a new file called `test_api_jwt.py`. Remember I am still in Agent mode.

```
import requests

BASE_URL = 'http://localhost:5000'
LOGIN_URL = f'{BASE_URL}/login'
DATA_URL = f'{BASE_URL}/data'

# Credentials for login (must exist in db.json)
credentials = {
    'username': 'Alice',
    'password': 'securepass'
}

# 1. Login to get JWT token
login_resp = requests.post(LOGIN_URL, json=credentials)
if login_resp.status_code != 200:
    print('Login failed:', login_resp.json())
    exit(1)
token = login_resp.json()['access_token']
headers = {'Authorization': f'Bearer {token}'}
print('JWT token:', token)

# 2. GET /data (protected)
get_resp = requests.get(DATA_URL, headers=headers)
print('\nGET /data response:')
print(get_resp.status_code, get_resp.json())

# 3. POST /data (protected)
new_doc = {
    'id': 999,
    'username': 'TestUser',
    'password': 'testpass'
}
post_resp = requests.post(DATA_URL, json=new_doc, headers=headers)
print('\nPOST /data response:')
print(post_resp.status_code, post_resp.json())
```

It suggested that I run this script along with the API but I did not know how, so I asked:

How do I run this script from my VS Code editor

It responded with the terminal commands:

```
python test_api_jwt.py or .venv/bin/python test_api_jwt.py
```

I chose the latter. It did not tell me to open a new terminal window, but I did anyway.

This is what my terminal window looked like after running these commands:

```
(.venv) axle@pc0482:~/Documents/python3-app$ .venv/bin/python test_api_jwt.py
JWT token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcmVzaCI6ZmFsc2UsIm1hdCI6MTc0ODIyMTI0OTcwIiwiaWF0Ij0wMDQwMjM0MTQyY2NjYy00MTkwLWI3MjEtZmM5ZGQ3NGJjMmUxIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6IkFsaWNlIiwibmJmIjojNzU4MjkyMjY4LCJjc3MjImJoimDExZTM2NWYtNjgzMCM0MTM5LTk5NWmtYWQ2YjU3YmMxNTIxIiwiaXZhbWwIjojNzU4MjkyMjY4fQ.IME-fq5Ld4FEJPAZZT6rM62Um0A1njd06UGz-4R0SoU

GET /data response:
200 [{ 'id': '1', 'password': '1234', 'username': 'Axle'}, { 'id': '2', 'password': '1234', 'username': 'Jane'}, { 'id': '3', 'password': '1234', 'username': 'Mary'}, { 'id': '492', 'password': 'secure pass', 'username': 'Alice'}, { 'id': '4', 'password': '1234', 'username': 'Bob'}, { 'id': '999', 'password': 'testpass', 'username': 'TestUser'}]

POST /data response:
201 { 'document': { 'id': '999', 'password': 'testpass', 'username': 'TestUser'}, 'message': 'Document added' }
```

Part 8

Is it possible to convert the existing Python application to a JavaScript based application. Well, just ask.

Can this application be converted to a JavaScript based application

Yes, this application can be converted to a JavaScript-based application. You can use:

Node.js with Express for the backend API (similar to Flask)
JSON Web Token (jsonwebtoken npm package) for JWT authentication
The file system (fs module) to read/write the JSON database
Mocha/Jest for testing

Would you like a step-by-step migration plan, a sample Express server, or help with a specific part of the conversion?

I asked for the step-by-step migration plan:

Here's a step-by-step migration plan to convert your Python Flask/JWT app to a JavaScript (Node.js/Express) application:

1. Set Up the Node.js Project

Create a new directory for your JS app.

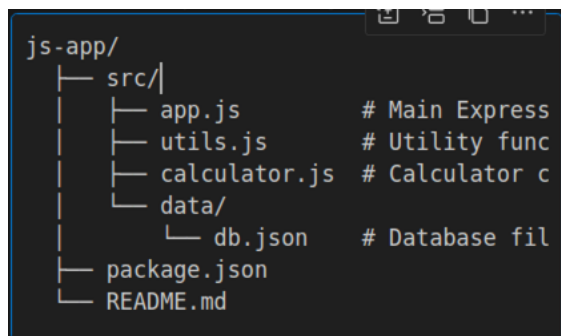
Run `npm init -y` to initialize.

Install dependencies:

```
npm install express jsonwebtoken body-parser fs
npm install --save-dev nodemon
```

2. Project Structure

Create folders/files similar to your Python app:



```
js-app/
├── src/
│   ├── app.js           # Main Express
│   ├── utils.js         # Utility func
│   ├── calculator.js    # Calculator c
│   └── data/
│       └── db.json      # Database fil
├── package.json
└── README.md
```

3. Implement the Express Server

In `src/app.js`, set up Express, JWT middleware, and endpoints:

`/login (POST):` Authenticate user, issue JWT

`/data (GET/POST):` Protected endpoints, require JWT

4. Implement Utility Functions

In `src/utils.js`, add functions for:

Reading/writing `db.json`

Generating random numbers (use `Math.random`)

5. Implement Calculator Module

In `src/calculator.js`, create a class with `add`, `subtract`, `multiply`, `divide` methods.

6. Copy and Adapt Data

Copy your `db.json` to `db.json`.

7. Testing

Use tools like Postman or write test scripts with `axios` or `supertest`.

8. Run the App

Use `node src/app.js` or `npx nodemon src/app.js` for development.

Would you like a sample Express server code to get started?

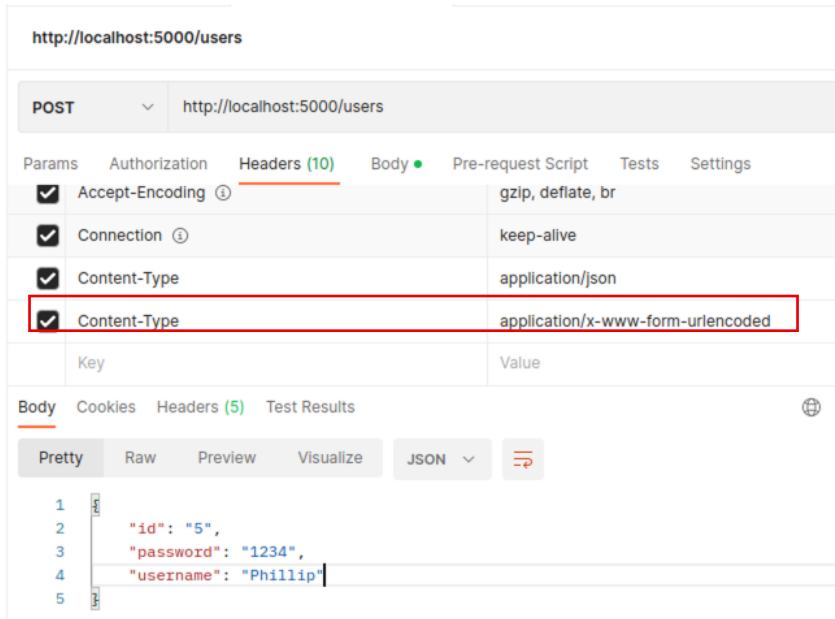
Notice that it is prompting us to get started with an Express server install.

Appendix A – Recreating the Python Programming environment

Sometimes a colleague may hand you a zipped Python program, or you want to start working with this bootcamp code but don't want to start from the beginning. Well, you can take any of the zipped files provided for this bootcamp and follow the instructions below. Remember these instructions are for a Linux VM, Windows and Mac may be slightly different.

1. Add this line to your requirements file if it is NOT already there: **werkzeug==2.3.7**
2. Create an environment via VS Code
3. Select requirements.txt when prompted in VS Code
4. Install the four packages in the requirements file
5. Run from this path: `/home/axle/Documents/python3-ap/.venv/bin/python src/main.py`
6. Skip tests for now
7. Here are the instructions on how to run the app after 1-5:
 - a. # Navigate to the project directory
 - b. `cd /home/axle/Documents/python3-ap`
 - c. # Run the main application
 - d. `/home/axle/Documents/python3-ap/.venv/bin/python src/main.py`

Appendix B – Using the Form in the Body tab of Postman



If you wanted to use the form under the Body tab in Postman, just add the new content-type as shown below.

Now you can just use the form instead of wrapping your data in curly braces:

